



# Currency Exchange Rate

Claude Code— Problem Statement

## 1. Problem Statement

Create a Currency Rate Monitoring Screen and corresponding APIs for various Countries and Currencies.

## 2. Technology Stack

Any technology stack of your choice may be used.

## 3. Features & Requirements

| Feature                                | Description, Business Rules & Design Constraints                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Currency DB &amp; Data Creation</b> | Create a currency_rate table and load the given country and currency information.                                                                                                                                                                                                                                                                                                                                                         |
| <b>Currency API</b>                    | <ol style="list-style-type: none"><li>1. Create an API to list all Countries and Currencies.</li><li>2. Calculate the exchange rate from the selected currencies.</li></ol>                                                                                                                                                                                                                                                               |
| <b>Validate Currencies</b>             | Validate the currencies selected. Only a limited set of currencies are allowed.                                                                                                                                                                                                                                                                                                                                                           |
| <b>Build UI Screens</b>                | <ol style="list-style-type: none"><li>3. Create a UI screen showing two currency dropdowns: Currency 1 and Currency 2.</li><li>4. When a currency is selected in Currency 1, remove it from the Currency 2 list.</li><li>5. Once both currencies are selected and submitted, fetch and display the exchange rate from the API.</li><li>6. Example: If USD → INR is stored, INR → USD must be calculated as the inverse in code.</li></ol> |
| <b>Test Cases</b>                      | Write test cases for the API and/or UI, covering both positive and negative scenarios.                                                                                                                                                                                                                                                                                                                                                    |

## 4. Key Points to Follow

### 4a. Database Table Structure

| Table         | Columns                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Currency      | <ul style="list-style-type: none"><li>id</li><li>currency_code</li><li>currency_name</li><li>country_name</li></ul> |
| Currency_rate | <ul style="list-style-type: none"><li>currency_code_from</li><li>currency_code_to</li><li>exchange_rate</li></ul>   |

### Seed Data — Currency table

```
{1, "INR", "Indian Rupees", "INDIA"}
{2, "USD", "US Dollars", "USA"}
{3, "CAD", "Canadian Dollars", "CANADA"}
{4, "EUR", "European Dollars", "EUROPE"}
{5, "AUD", "Australian Dollars", "AUSTRALIA"}
{6, "AED", "UAE Dirham", "UAE"}
```

### Seed Data — Currency\_rate table

```
{"USD", "INR", 80.08}
{"CAD", "INR", 61.62}
{"USD", "CAD", 1.36}
{"EUR", "INR", 93.14}
{"AUD", "INR", 56.81}
{"AED", "INR", 22.79}
```

### 4b. API Specification

#### GET /currency

Returns a list of all currencies and countries.

```
{
  "INR": { "countryName": "India", "currencyCode": "INR", "currencyName": "Indian Rupees" },
  "USD": { "countryName": "USA", "currencyCode": "USD", "currencyName": "US Dollars" }
}
```

### GET /exchange-rate/{fromCur}/{toCur}

Returns the exchange rate between two specified currencies.

```
{
  "fromCurrencyCode": "USD",
  "toCurrencyCode": "INR",
  "exchangeRate": "80.08"
}
```

## 4c. UI / UX (Sample Reference)

The following is a sample layout for reference only. Participants may design their own interface.

1 United States Dollar equals

80.08 Indian Rupee

14 Jul, 2:00 PM UTC - Disclaimer

|       |                        |
|-------|------------------------|
| 1     | United States Dollar ▼ |
| 80.08 | Indian Rupee ▼         |

## 5. Expected Output (via Claude Code)

All items below must be generated and documented using Claude Code:

- Associates are expected to write .md files for custom instructions on code logic, test cases (if any) and code review.
- GHCP commands/prompts used — store in a text file or within the checked-in code.
- Database scripts — table creation scripts.
- Scripts to load/seed tables.
- Working API code.
- Working UI code.
- Unit test cases.
- Screenshots of the output with visible date and timestamp.

## 6. Instructions

---

### 6.1 Development

- Fully develop the use case using Claude commands.
- Associates are expected to write .md files for custom instructions on code logic, test cases (if any) and code review.

### 6.2 Documentation of Commands & Prompts

- Document all Claude commands and prompts used, in a text file.

Commands are direct instructions to perform actions; prompts are inputs that guide generation of responses or outputs.

### 6.3 Version Control

- Maintain a history of Copilot commands and generated code using GitLab. (Use GitLab if you have access)

### 6.4 Database Components

- Use Claude Code to generate and document all database components.

### 6.5 Unit Test Cases

- Use Claude to generate and document unit test cases.
- Ensure both positive and negative test cases are covered.

### 6.6 Comprehensive Project Documentation

- Use Claude Code to generate complete documentation for all code components.
- Include a brief project note for offline evaluation covering:
  - Reasons behind the choice of case study and framework used.
  - How prompts were used to ensure end-to-end functionality.
  - Code documentation, unit test coverage, and any additional notes.

## 6.7 Final Product

- Ensure the final product is fully functional with all features implemented.
- Capture a screenshot of the UI displaying the final output.